

PROCEDIMENTO OPERACIONAL PADRÃO PARA EXTRAÇÃO E ANÁLISE DE DADOS EM BANCO DE DADOS APLICADO À ENGENHARIA DE PRODUÇÃO

Gabriel Costa de Andrade Sodré (Universidade Federal de Goiás-UFG))

Luís Humberto Franco de Carvalho (Universidade Federal de Goiás-UFG)



A crescente digitalização dos processos industriais, impulsionada pela Indústria 4.0, exige que o engenheiro de produção desenvolva competências analíticas baseadas no acesso direto a dados operacionais. Nesse contexto, este trabalho tem como objetivo elaborar um Procedimento Operacional Padrão (POP), no formato de artigo científico, para orientar a extração e análise de dados diretamente de um banco de dados. A metodologia adotada envolve a revisão teórica sobre Sistemas de Informação Gerencial (SIG), arquitetura de software e bancos de dados, incluindo modelagem relacional, normalização e tipos de relacionamentos. Além disso, é proposto um procedimento estruturado para conexão, consulta e análise de dados em banco relacional e não relacional. Como resultado, apresenta-se um modelo conceitual e prático que permite ao engenheiro de produção obter autonomia analítica, transformando dados brutos em informações relevantes para tomada de decisão. Conclui-se que o acesso direto ao banco de dados aumenta a eficiência e a confiabilidade das análises no ambiente produtivo.

Palavras-chave: Engenharia de Produção, Banco de Dados; POP; Indústria 4.0.

1. Introdução

Nas organizações modernas, com o avanço da Indústria 4.0, as organizações passaram a adotar sistemas computacionais integrados para melhorar a eficiência de seus processos produtivos e administrativos. A digitalização das operações permitiu maior controle, redução de desperdícios e tomada de decisão baseada em dados (SLACK; CHAMBERS; JOHNSTON, 2009).

O controle, a produção e tomada de decisão de sistemas produtivos dependem diretamente da qualidade e disponibilidade das informações. No entanto, frequentemente, os dados podem chegar aos profissionais por meio de relatórios e dashboards, podendo estar filtrados, atrasados ou agregados. Nesse contexto, o acesso direto ao banco de dados torna-se um diferencial competitivo para o engenheiro de produção para melhorar a qualidade de dados.

Para acessar e extrair os dados de uma empresa é necessário entender e compreender como está a mesma está organizada e como o seu sistema de informação gerencial funciona, como acontece a conexão com outros sistemas ou subsistemas como: ERP (Enterprise Resource Planning), MES (Manufacturing Execution System), WMS (Warehouse Management System), SCADA (Supervisory Control and Data Acquisition), CRM (Customer Relationship Management) e ferramentas de BI (Business Intelligence). Slack, Chambers e Johnston (2009) destacam que a integração de informações impacta diretamente a eficiência operacional. De acordo com Laudon e Laudon (2014), sistemas de informação são essenciais para suportar processos organizacionais e decisões estratégicas.

Assim, este trabalho busca integrar conceitos de sistemas de informação, banco de dados e arquitetura de software, com foco na aplicação prática para engenheiros de produção, produzindo um POP(Procedimento Operacional Padrão) de acesso a banco de dados a partir de uma situação prática e posterior análise dos dados e suas relações.

2. Revisão de Literatura

2.1 Procedimento Operacional Padrão (POP)

Diante da complexidade que envolve a compreensão da estrutura relacional e a sintaxe de consulta, torna-se evidente a necessidade de ferramentas de padronização corporativa, como a elaboração de um Procedimento Operacional Padrão (POP). O POP atua como um documento normativo e instrucional que detalha e sistematiza as melhores práticas para a execução de um fluxo de trabalho (LAUDON; LAUDON, 2014).

Ao aplicar um POP voltado especificamente para a conexão, extração e análise via SQL, a organização industrial democratiza o acesso seguro à informação estruturada. Esse procedimento não apenas mitiga erros de consulta que poderiam sobrecarregar os servidores, mas também capacita o engenheiro de produção a conquistar autonomia analítica, consolidando uma cultura de tomada de decisão baseada em evidências sólidas.

2.2 Linguagem de Modelagem Unificada (UML)

A estruturação eficiente desses bancos de dados, no entanto, exige um planejamento arquitetônico prévio, frequentemente apoiado pela UML (Linguagem de Modelagem Unificada). A UML fornece um vocabulário padronizado e ferramentas visuais — com destaque para o diagrama de classes — que permitem traduzir os complexos requisitos do sistema em uma estrutura lógica e perfeitamente compreensível (FOWLER, 2004).

O uso do diagrama de classes é de suma importância na modelagem de dados, pois ele mapeia de forma esquemática as tabelas, definindo seus atributos, tipos de dados e propriedades limitantes. Dessa forma, a UML atua como uma ponte analítica entre a visão de processos do engenheiro e a representação digital estrita exigida pelo sistema computacional.

A partir desse mapeamento conceitual fornecido pela UML, estabelece-se o procedimento de relacionamento entre entidades, uma etapa crítica para assegurar a integridade do banco de dados relacional. De acordo com Date (2004), o modelo relacional organiza as informações em tabelas distintas (entidades) que se conectam por meio de chaves primárias (identificadores únicos de um registro) e chaves estrangeiras (atributos que referenciam outras tabelas).

Essas conexões lógicas determinam as regras de cardinalidade — categorizadas em um-para-um (1:1), um-para-muitos (1:N) e muitos-para-muitos (N:M) —, orquestrando como os dados de diferentes setores industriais se cruzam. A modelagem correta desses relacionamentos garante a normalização do banco, eliminando anomalias de atualização e redundâncias que poderiam distorcer os indicadores de desempenho produtivos.

2.3 Linguagem Relacional SQL (*Structured Query Language*)

Com a infraestrutura de dados devidamente modelada, normalizada e inter-relacionada, a etapa de extração e manipulação passa a ser executada por meio da linguagem SQL (*Structured Query Language*). O SQL consolidou-se como a linguagem padrão universal para a comunicação com bancos de dados relacionais, permitindo aos usuários realizar desde seleções simples de colunas até *joins* (junções) complexos que agrupam múltiplos conjuntos de dados simultaneamente (ELMASRI; NAVATHE, 2011).

2.4 Linguagem Não relacional (NoSQL)

Em contrapartida à estrutura rígida e normalizada dos modelos relacionais, a crescente necessidade de processar e armazenar grandes volumes de dados não estruturados ou semiestruturados impulsionou a adoção dos bancos de dados NoSQL (*Not Only SQL*).

Esse modelo consolidou-se sendo altamente flexível e de fácil escalabilidade horizontal para aplicações modernas, permitindo aos desenvolvedores manipular informações em formatos diversos — como documentos, chave-valor, grafos ou famílias de colunas — sem a exigência de um esquema pré-definido com a disponibilidade e desempenho em ambientes distribuídos (SADALAGE; FOWLER, 2013).

2.5- Sistemas de Informação (SI) e SIG

Os Sistemas de Informação (SI) são conjuntos organizados de pessoas, processos e tecnologia que coletam, processam e distribuem informações. O Sistema de Informação Gerencial (SIG) é responsável por apoiar decisões táticas e estratégicas (LAUDON; LAUDON, 2014).

2.6 ERP (Enterprise Resource Planning)

O ERP é um sistema de gestão empresarial que integra diferentes áreas da organização, como finanças, recursos humanos, compras, vendas e estoque. Segundo Davenport (1998), o ERP permite a centralização das informações, promovendo maior controle e padronização dos processos. Laudon e Laudon (2014) destacam que esses sistemas são fundamentais para a integração organizacional e para a melhoria da eficiência operacional.

2.7 MES (Manufacturing Execution System)

O MES é responsável pelo gerenciamento e monitoramento da produção em tempo real. Ele atua diretamente no chão de fábrica, coletando dados e permitindo o controle da produção, qualidade e rastreabilidade. De acordo com O'Brien (2010), sistemas de execução da manufatura são essenciais para garantir a visibilidade e o controle das operações produtivas.

2.8 WMS (Warehouse Management System)

O WMS é utilizado para gerenciar operações de armazenagem, incluindo controle de estoque, movimentação de materiais e otimização de espaços. Sua aplicação é fundamental em centros de distribuição, contribuindo para maior eficiência logística (SLACK; CHAMBERS; JOHNSTON, 2009).

2.9 SCADA (Supervisory Control and Data Acquisition)

O SCADA é um sistema utilizado para monitorar e controlar processos industriais em tempo real, permitindo a coleta de dados diretamente de sensores e equipamentos. Segundo Laudon e Laudon (2014), sistemas de controle e supervisão são essenciais para a automação industrial e para a tomada de decisão operacional.

2.10 CRM (Customer Relationship Management)

O CRM é um sistema voltado para a gestão do relacionamento com clientes, auxiliando nas áreas de vendas, marketing e atendimento. Esses sistemas permitem melhor compreensão das necessidades dos clientes e aumento da competitividade organizacional (O'BRIEN, 2010).

2.10 BI / Analytics

As ferramentas de BI são utilizadas para análise de dados, geração de relatórios e apoio à tomada de decisão estratégica. Segundo Laudon e Laudon (2014), o uso de sistemas de apoio à decisão permite transformar dados em informações relevantes para a gestão.

2.11 Banco de Dados e Estrutura de Dados

Os bancos de dados são responsáveis pelo armazenamento estruturado das informações. Podem ser classificados como:

- Relacionais (SQL)
- Não relacionais (NoSQL)

A modelagem envolve conceitos como normalização (1FN–3FN), relacionamentos (1:1, 1:N, N:M) e integridade dos dados (GIL, 2010).

2.12 Arquitetura de Software e integração de sistemas industriais

A arquitetura de software corresponde à forma como os sistemas computacionais são organizados, estruturados e integrados. Ela define como os componentes do sistema se relacionam, como os dados circulam e como ocorre a comunicação entre as diferentes partes da aplicação. Segundo Ian Sommerville (2011), a arquitetura de software representa a estrutura fundamental de um sistema, incluindo seus componentes, relacionamentos e princípios de organização.

No contexto empresarial e industrial, a arquitetura de software é essencial para garantir escalabilidade, manutenção, segurança e integração entre sistemas como ERP, MES, WMS, SCADA e ferramentas de Business Intelligence. Além disso, arquiteturas modernas buscam desacoplar os componentes do sistema, permitindo maior flexibilidade e facilidade de atualização tecnológica.

Os principais modelos de arquitetura utilizados atualmente são MVC (Model-View-Controller), arquitetura em camadas e Clean Architecture.

A arquitetura MVC divide o sistema em três componentes principais:

- **Model (Modelo):** responsável pelos dados e regras de negócio;
- **View (Visão):** responsável pela interface com o usuário;
- **Controller (Controlador):** faz a comunicação entre interface e lógica do sistema.

Vantagens:

- Separação entre interface e lógica de negócio;
- Facilidade de manutenção;
- Reutilização de componentes;
- Desenvolvimento mais organizado.

Desvantagens:

- Maior complexidade em sistemas pequenos;
- Necessidade de maior planejamento;
- Pode gerar excesso de arquivos e componentes.

Esse modelo é amplamente utilizado em sistemas web e aplicações corporativas.

A arquitetura em camadas organiza o sistema em diferentes níveis funcionais, normalmente:

1. Camada de apresentação;
2. Camada de aplicação;
3. Camada de negócios;
4. Camada de dados.

Nesse modelo, o banco de dados fica localizado na camada de infraestrutura, responsável pelo armazenamento e recuperação das informações.

Vantagens:

- Organização modular;
- Facilidade de manutenção;
- Maior segurança;
- Possibilidade de atualização isolada das camadas.

Desvantagens:

- Comunicação entre camadas pode reduzir desempenho;
- Maior consumo de recursos;
- Complexidade em sistemas muito grandes.

Essa arquitetura é bastante utilizada em sistemas ERP e aplicações industriais.

A Clean Architecture, proposta por Robert C. Martin, organiza o sistema de forma desacoplada, priorizando independência entre regras de negócio, interface e banco de dados.

Nesse modelo, o banco de dados é tratado apenas como um detalhe de infraestrutura, permitindo que o sistema funcione independentemente da tecnologia utilizada.

Vantagens:

- Alto desacoplamento;
- Facilidade de testes;
- Maior flexibilidade tecnológica;
- Facilidade de manutenção e evolução.

Desvantagens:

- Estrutura mais complexa;
- Curva de aprendizado elevada;
- Maior tempo inicial de desenvolvimento.

A Clean Architecture é bastante utilizada em sistemas modernos que necessitam alta escalabilidade e integração.

O modelo de arquitetura hexagonal, também conhecido como **Ports and Adapters**, foi proposto por Alistair Cockburn com o objetivo de reduzir o acoplamento entre a lógica de negócio e os elementos externos do sistema, como banco de dados, interfaces gráficas, APIs e serviços externos.

Nesse modelo, o núcleo da aplicação contém apenas as regras de negócio, enquanto os elementos externos se conectam ao sistema por meio de “portas” (ports) e “adaptadores” (adapters). Dessa forma, a lógica principal permanece independente das tecnologias utilizadas.

A arquitetura recebe o nome “hexagonal” porque normalmente é representada graficamente em formato de hexágono, indicando que o sistema pode se conectar a diferentes interfaces externas sem depender diretamente delas.

Na arquitetura hexagonal:

- O centro do sistema contém as regras de negócio;
- As portas definem contratos de comunicação;
- Os adaptadores fazem a conexão com tecnologias externas.

Exemplo:

- Banco de dados;
- APIs;
- Interfaces web;

- Sistemas ERP;
- Sensores industriais.

Nesse modelo, o banco de dados é tratado como um componente externo, semelhante à Clean Architecture.

Vantagens da Arquitetura Hexagonal

- Alto desacoplamento: a lógica de negócio fica independente do banco de dados e das interfaces;
- Facilidade de testes: é possível testar as regras de negócio sem precisar conectar ao banco real;
- Flexibilidade tecnológica: permite trocar tecnologias sem alterar o núcleo do sistema;
- Facilidade de integração: muito útil em sistemas industriais integrados com ERP, MES, SCADA e APIs externas;
- Melhor manutenção: mudanças externas afetam menos a aplicação principal.

Desvantagens da Arquitetura Hexagonal:

- Maior complexidade estrutural :exige mais planejamento e organização;
- Curva de aprendizado elevada : pode ser difícil para equipes iniciantes;
- Mais arquivos e abstrações: o sistema pode ficar mais extenso;
- Desenvolvimento inicial mais lento: a separação das responsabilidades exige maior tempo de implementação.

Na Engenharia de Produção, a arquitetura hexagonal é importante porque facilita a integração entre diferentes sistemas industriais e corporativos. Um sistema pode, por exemplo:

- receber dados do MES;
- armazenar informações em PostgreSQL;
- enviar indicadores para ferramentas de BI;
- integrar-se ao ERP da empresa.

Tudo isso sem alterar as regras centrais do sistema.

Essa abordagem favorece escalabilidade, rastreabilidade e interoperabilidade dos sistemas produtivos

A integração entre ERP, MES, SCADA, WMS e CRM permite uma visão sistêmica da organização. O BI atua transformando dados em informação estratégica (O'BRIEN, 2010).

2.12 Strings e Drivers

Drivers são bibliotecas ou softwares responsáveis por estabelecer a comunicação entre uma aplicação e o Sistema Gerenciador de Banco de Dados (SGBD).

Eles funcionam como “tradutores”, permitindo que linguagens de programação, sistemas ERP, ferramentas de BI ou aplicações industriais consigam enviar comandos ao banco de dados e receber respostas.

Cada banco de dados possui drivers específicos.

Tabela 1: Drivers e Bancos

Banco de Dados	Driver
PostgreSQL	psycopg2
MySQL	mysql-connector
MongoDB	pymongo
SQL Server	ODBC / pyodbc

Fonte: Os autores

Quando um sistema em Python deseja acessar o PostgreSQL, ele utiliza o driver psycopg2 para criar a conexão.

A string de conexão é um conjunto de parâmetros utilizados para informar ao driver como acessar o banco de dados.

Ela contém informações como:

- endereço do servidor;
- porta;
- nome do banco;
- usuário;
- senha.

A string funciona como um “endereço completo” do banco de dados.

Figura 1: Modelo de string para o acesso ao Banco de Dados

PostgreSQL

```
host=localhost  
port=5432  
dbname=producao  
user=admin  
password=123
```

Fonte: Autoria Própria

O processo normalmente ocorre da seguinte forma:

Aplicação → Driver → String de conexão → Banco de Dados

1. A aplicação utiliza um driver;
2. O driver lê a string de conexão;

3. O sistema localiza o banco;
4. A conexão é estabelecida;
5. Queries podem ser executadas.

No contexto industrial, drivers e strings permitem que sistemas como:

- ERP;
- MES;
- SCADA;
- WMS;
- ferramentas de BI;

acessem dados produtivos em tempo real.

Isso possibilita:

- monitoramento da produção;
- geração de indicadores;
- rastreabilidade;
- integração entre sistemas.

Segundo Ramez Elmasri e Shamkant Navathe, os mecanismos de conexão permitem que aplicações realizem operações de armazenamento, consulta e manipulação de dados em sistemas gerenciadores de bancos de dados, garantindo interoperabilidade entre software e infraestrutura de dados.

3. Método de Pesquisa

A presente pesquisa classifica-se como de natureza aplicada, com abordagem qualitativa e caráter exploratório, voltada à resolução de um problema prático no ambiente

industrial. O objetivo central consistiu em traduzir conceitos complexos de arquitetura da informação para a realidade tática e operacional da Engenharia de Produção, culminando na elaboração de um Procedimento Operacional Padrão (POP). Diante do cenário de transformação digital exigido pela Indústria 4.0, a metodologia foi estruturada para não apenas descrever ferramentas de software, mas para integrar a tecnologia à rotina do engenheiro, transformando o acesso direto a dados operacionais em um pilar estratégico para a tomada de decisão ágil e devidamente fundamentada.

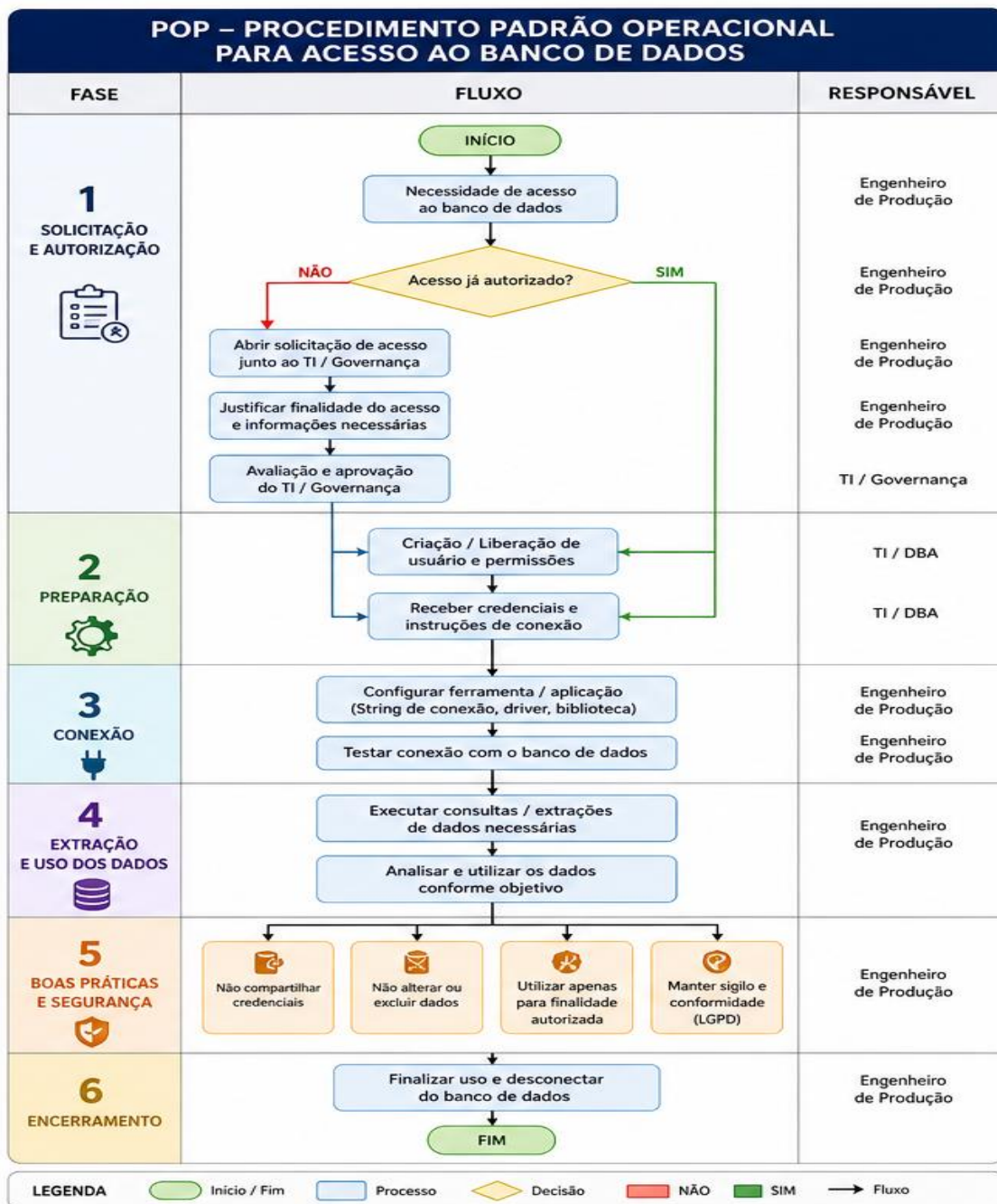
A primeira etapa metodológica baseou-se em uma revisão bibliográfica sistemática, que serviu como alicerce para a construção do procedimento. As buscas abordaram temáticas cruciais como Sistemas de Informação Gerencial (SIG), arquitetura de bancos de dados relacionais, Linguagem de Modelagem Unificada (UML) e a sintaxe da linguagem. Essa imersão teórica permitiu o mapeamento das melhores práticas de mercado e das normas estritas de integridade referencial. Dessa forma, garantiu-se o rigor científico necessário para sustentar a criação de um documento normativo que dialogue diretamente com as exigências de confiabilidade, rastreabilidade e segurança da informação presentes nas operações de manufatura modernas.

Na segunda etapa, o foco metodológico voltou-se para a estruturação lógica do sistema de banco de dados, avaliando criticamente os fatores que transcendem a mera operação de um software. Utilizando os princípios da UML — especificamente os diagramas de classes e os procedimentos de relacionamento entre entidades —, delineou-se como o modelo conceitual deve ser lido e interpretado por um profissional não originário da área de Tecnologia da Informação (TI). Essa fase enfatizou que a eficácia analítica depende intrinsecamente de como a arquitetura da informação reflete os processos reais de fábrica e de como os dados são normalizados. Somente com essa base estrutural bem compreendida é possível evitar anomalias nas consultas e assegurar que as métricas extraídas representem o estado real do ecossistema produtivo.

Por fim, a terceira e última etapa consistiu na consolidação dessas diretrizes teóricas e arquitetônicas no desenvolvimento do POP propriamente dito. O documento foi projetado para atuar como uma interface facilitadora e utilizável, padronizando as rotinas de conexão, formulação de *queries* e extração de relatórios. Mais do que um manual técnico, o POP foi concebido como uma ferramenta de empoderamento, minimizando riscos de

sobrecarga nos servidores e garantindo a autonomia analítica do engenheiro de produção. Para sintetizar a aplicação prática dessa metodologia na rotina de análise, estruturou-se um fluxograma operacional detalhando as cinco etapas essenciais para o uso eficiente do banco de dados no processo decisório.

Figura 2: Etapas necessárias para o acesso ao Banco de Dados



Fonte: Autoria própria

4. Resultados

A discussão teórica e prática gerou o mapa mental e fluxograma

REFERÊNCIAS

- COCKBURN, Alistair. *Hexagonal Architecture*. 2005. Disponível em: [Alistair Cockburn Official Website](#). Acesso em: 12 maio 2026.
- DATE, C. J. **Introdução a Sistemas de Bancos de Dados**. 8. ed. Rio de Janeiro: Elsevier, 2004.
- DAVENPORT, Thomas H. **Putting the enterprise into the enterprise system**. Harvard Business Review, Boston, v. 76, n. 4, p. 121–131, jul./ago. 1998.
- ELMASRI, R.; NAVATHE, S. B. **Sistemas de Banco de Dados**. 6. ed. São Paulo: Pearson Addison Wesley, 2011.
- FOWLER, M. **UML Essencial: um breve guia para a linguagem padrão de modelagem de objetos**. 3. ed. Porto Alegre: Bookman, 2004.
- LAUDON, K. C.; LAUDON, J. P. **Sistemas de Informação Gerenciais**. 11. ed. São Paulo: Pearson Prentice Hall, 2014.
- MARTIN, Robert C. **Clean Architecture: A Craftsman's Guide to Software Structure and Design**. Boston: Prentice Hall, 2017.
- O'BRIEN, James A. **Sistemas de Informação e as Decisões Gerenciais na Era da Internet**. 3. ed. São Paulo: Saraiva, 2010.
- PRESSMAN, Roger S. **Engenharia de Software**. 8. ed. Porto Alegre: AMGH, 2016.
- SADALAGE, Pramod J.; FOWLER, Martin. **NoSQL Essencial: um guia conciso para o mundo emergente da persistência poliglota**. 1. ed. São Paulo: Novatec, 2013.

SCHWAB, K. **A Quarta Revolução Industrial**. São Paulo: Edipro, 2016.

SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. **Sistema de Banco de Dados**. 6. ed. Rio de Janeiro: Elsevier, 2012.

SLACK, Nigel; CHAMBERS, Stuart; JOHNSTON, Robert. **Administração da Produção**. 3. ed. São Paulo: Atlas, 2009.

SOMMERVILLE, Ian. **Engenharia de Software**. 9. ed. São Paulo: Pearson, 2011.